

2-LAB

Normalization

1. We have table called “Booking_receipt” with the following attributes

booking_id	Passenger_full_name	passenger_passport_number	flight_number	departure_airport_name	departure_city	arrival_airport_name	arrival_city	airline_name	seat_numbers	ticket_price
1	Togzhan Berikbolova	P123	F123	Airport Almaty	Almaty	Airport Astana	Astana	Air Astana	12A, 12B	20000

Check 1NF: every cell not contain one value, for exemple: seat_numbers = “12A, 12B”, there are two values in one cell. The table violates 1NF

Check 2NF: When we split the seat numbers into separate rows to fix 1NF, the primary key becomes the composite key (booking_id, seat_number). However, attributes like flight_number, departure_airport_name, and ticket_price depend only on booking_id, not on the full composite key (booking_id, seat_number). This creates partial dependencies, so the table violates 2NF.

Check 3NF: in the 2NF Booking table, the primary key is booking_id. However, some non-key attributes depend on other non-key attributes. For example, departure_city depends on departure_airport_name, and arrival_city depends on arrival_airport_name. This creates transitive dependencies.

2.

Normalize to 1NF:

To convert the table to 1NF, we make sure that every cell contains only one value. In the original table, seat_numbers contains multiple values, for example, "12A, 12B", so it violates 1NF.

We separate the seat numbers into different rows. Now each row contains only one seat_number and every cell has a single value.

The primary key is the composite key (booking_id, seat_number). There are no foreign keys.

booking_id	Passenger_full_name	passenger_passport_number	flight_number	departure_airport_name	departure_city	arrival_airport_name	arrival_city	airline_name	seat_number	ticket_price
1	Togzhan Berikbolova	P123	F123	Airport Almaty	Almaty	Airport Astana	Astana	Air Astana	12A	20000
1	Togzhan Berikbolova	P123	F123	Airport Almaty	Almaty	Airport Astana	Astana	Air Astana	12B	20000

Primary key: booking_id, seat_number

Foreign key: none

Normalize to 2NF:

To convert the table to 2NF, we remove the partial dependency. In 1NF, the primary key is a composite key (booking_id, seat_number). Most attributes depend only on booking_id, while seat_number depends on the combination of booking_id and seat_number.

Therefore, we divide the table into two tables: Booking and Booking_seat. Booking stores the information related to the booking, and booking_id is its primary key.

Booking_seat stores the seat numbers for each booking. Its composite primary key is (booking_id, seat_number), and booking_id is also a foreign key that references Booking(booking_id).

Booking

Primary key: booking_id

Booking_id	Passenger_full_name	passenger_passport_number	flight_number	departure_airport_name	departure_city	arrival_airport_name	arrival_city	airline_name	ticket_price
1	Togzhan Berikbolova	P123	F123	Airport Almaty	Almaty	Airport Astana	Astana	Air Astana	20000

Booking_seat

Primary key: booking_id, seat_number

Foreign key: booking_id → Booking(booking_id)

Booking_id	seat_number
1	12A
1	12B

Normalization 3NF:

To convert the tables to 3NF, we remove transitive dependencies. Some non-key attributes depend on other non-key attributes. For example, passenger_full_name depends on passenger_passport_number, flight information depends on flight_number, and city depends on airport_name.

Therefore, we separate the data into Booking, Passenger, Flight, Airport, and Booking_seat tables. Now the non-key attributes in each table depend on the primary key of that table.

Booking

Primary key: booking_id;

Foreign key: passenger_passport_number, flight_number

booking_id	passenger_passport_number	flight_number	ticket_price
------------	---------------------------	---------------	--------------

1	P123	F123	20000
---	------	------	-------

Passenger

Primary key: passenger_passport_number

Foreign key: none

Passenger_passport_number	Passenger_full_name
P123	Togzhan Berikbolova

Flight

Primary key: flight_number

Foreign key: departure_airport_name, arrival_airport_name

flight_number	departure_airport_name	arrival_airport_name	airline_name
F123	Airport Almaty	Airport Astana	Air Astana

Airport

Primary key: airport_name

Foreign key: none

Airport_name	City
Airport Almaty	Almaty
Airport Astana	Astana

Booking_seat

Primary key: booking_id, seat_number

Foreign key: booking_id → Booking(booking_id)

booking_id	seat_number
1	12A
1	12B

3.

Functional Dependencies

For 1NF:

The seat_numbers column has more than one value, for example, "12A, 12B". In 1NF, we put each seat number in a separate row.

The row is identified by booking_id and seat_number.

(booking_id, seat_number) → passenger_passport_number, flight_number, ticket_price

For 2NF:

Some attributes depend only on booking_id:

booking_id → passenger_passport_number

booking_id → flight_number

booking_id → ticket_price

Because of this, we separate the data into Booking and Booking_seat tables.

For 3NF:

Some attributes depend on other attributes:

passenger_passport_number → passenger_full_name

flight_number → departure_airport_name

flight_number → arrival_airport_name

flight_number → airline_name

departure_airport_name → departure_city

arrival_airport_name → arrival_city

Because of these dependencies, we separate the data into Booking, Passenger, Flight, Airport, and Booking_seat tables.

4. Check Flights table for 3NF

The primary key of the Flights table is flight_id.

The functional dependency is:

flight_id → sch_departure_time, sch_arrival_time, departing_airport_id, arriving_airport_id, departing_gate, arriving_gate, airline_id, act_departure_time, act_arrival_time, created_at, updated_at

It means that flight_id identifies one flight and its information.

As we can see the Flights table does not store information about airports, such as airport_name, city, country and state. This information is stored in the Airport table.

The Flights table also does not store airline_name or airline_country. This information is stored in the Airline table.

So, there is no transitive dependency between non-key attributes inside the Flights table.

Therefore, the Flights table is in 3NF.

5.

pgAdmin 4 Object Tools Edit View Window Help

pgAdmin 4

Query Query History

```
3 passenger_full_name varchar(100) not null
4 );
5
6 create table Airport(
7     airport_name varchar(50) primary key,
8     city varchar(50) not null
9 );
10
11 create table Flight(
12     flight_number varchar(20) primary key,
13     departure_airport_name varchar(50) not null references Airport(airport_name),
14     arrival_airport_name varchar(50) not null references Airport(airport_name),
15     airline_name varchar(50) not null
16 );
17
18 create table Booking(
19     booking_id serial primary key,
20     passenger_passport_number varchar(20) not null references Passenger(passenger_passport_number),
21     flight_number varchar(20) not null references Flight(flight_number),
22     ticket_price decimal(10, 2) not null
23 );
24
25 create table Booking_seat(
26     booking_id int references Booking(booking_id),
27     seat_number varchar(10) not null
28 )
29
30
```

Data Output Messages Notifications

CREATE TABLE

Query returned successfully in 47 msec.

✓ Query returned successfully in 47 msec. ✕

Total rows: Query complete 00:00:00.047 LF Ln 14, Col 59

pgAdmin 4 Object Tools Edit View Window Help

pgAdmin 4

Query Query History

```
5 ('P123', 'Nura Kulaimov'),
6 ('P124', 'Aruzhan Aikyn'),
7 ('P125', 'Dana Kim');
8
9 insert into Airport(airport_name, city)
10 values
11 ('Airport Almaty', 'Almaty'),
12 ('Airport Astana', 'Astana'),
13 ('Airport Shymkent', 'Shymkent'),
14 ('Airport Aktau', 'Aktau'),
15 ('Airport Atyrau', 'Atyrau');
16
17 insert into Flight(flight_number, departure_airport_name, arrival_airport_name, airline_name)
18 values
19 ('F121', 'Airport Almaty', 'Airport Astana', 'Air Astana'),
20 ('F122', 'Airport Astana', 'Airport Almaty', 'Air Astana'),
21 ('F123', 'Airport Almaty', 'Airport Shymkent', 'SCAT'),
22 ('F124', 'Airport Shymkent', 'Airport Astana', 'SCAT'),
23 ('F125', 'Airport Astana', 'Airport Aktau', 'Air Astana');
24
25 insert into Booking(passenger_passport_number, flight_number, ticket_price)
26 values
27 ('P121', 'F121', 20000),
28 ('P122', 'F122', 22000),
29 ('P123', 'F123', 18000),
30 ('P124', 'F124', 19000),
31 ('P125', 'F125', 21000);
32
33 insert into Booking_seat(booking_id, seat_number)
34 values
35 (1, '12A'),
36 (2, '15R');
37
```

Data Output Messages Notifications

INSERT 0 5

Query returned successfully in 62 msec.

✓ Query returned successfully in 62 msec. ✕

Total rows: Query complete 00:00:00.062 LF Ln 17, Col 39

Object Explorer

- PRC8
- TSIS1
- airport_db
 - Casts
 - Catalogs
 - Event Triggers
 - Extensions
 - Foreign Data Wrappers
 - Languages
 - Publications
 - Schemas (1)
 - public
 - Aggregates
 - Collations
 - Domains
 - FTS Configurations
 - FTS Dictionaries
 - FTS Parsers
 - FTS Templates
 - Foreign Tables
 - Functions
 - Materialized Views
 - Operators
 - Procedures
 - Sequences
 - Tables (5)
 - airport
 - booking
 - booking_seat
 - flight
 - passenger
 - Trigger Functions
 - Types
 - Views
 - Subscriptions
 - javaslive6

airport_db/postgres@PostgreSQL 17*

airport_db/postgres@PostgreSQL 17

Query Query History

```
1 select * from Flight;
```

Data Output Messages Notifications

Showing rows: 1 to 5 Page No: 1 of 1

	flight_number [PK] character varying (20)	departure_airport_name character varying (50)	arrival_airport_name character varying (50)	airline_name character varying (50)
1	F121	Airport Almaty	Airport Astana	Air Astana
2	F122	Airport Astana	Airport Almaty	Air Astana
3	F123	Airport Almaty	Airport Shymkent	SCAT
4	F124	Airport Shymkent	Airport Astana	SCAT
5	F125	Airport Astana	Airport Aktau	Air Astana

✓ Successfully run. Total query runtime: 64 msec. 5 rows affected. ✕

Total rows: 5 Query complete 00:00:00.064 LF Ln 1, Col 22

Object Explorer

- PRC8
- TSIS1
- airport_db
 - Casts
 - Catalogs
 - Event Triggers
 - Extensions
 - Foreign Data Wrappers
 - Languages
 - Publications
 - Schemas (1)
 - public
 - Aggregates
 - Collations
 - Domains
 - FTS Configurations
 - FTS Dictionaries
 - FTS Parsers
 - FTS Templates
 - Foreign Tables
 - Functions
 - Materialized Views
 - Operators
 - Procedures
 - Sequences
 - Tables (5)
 - airport
 - booking
 - booking_seat
 - flight
 - passenger
 - Trigger Functions
 - Types
 - Views
 - Subscriptions
 - javaslive6

airport_db/postgres@PostgreSQL 17*

airport_db/postgres@PostgreSQL 17

Query Query History

```
1 select * from Airport;
```

Data Output Messages Notifications

Showing rows: 1 to 5 Page No: 1 of 1

	airport_name [PK] character varying (50)	city character varying (50)
1	Airport Almaty	Almaty
2	Airport Astana	Astana
3	Airport Shymkent	Shymkent
4	Airport Aktau	Aktau
5	Airport Atyrau	Atyrau

✓ Successfully run. Total query runtime: 74 msec. 5 rows affected. ✕

Total rows: 5 Query complete 00:00:00.074 LF Ln 1, Col 23

Object Explorer

- PRC8
- TSIS1
- airport_db
 - Casts
 - Catalogs
 - Event Triggers
 - Extensions
 - Foreign Data Wrappers
 - Languages
 - Publications
 - Schemas (1)
 - public
 - Aggregates
 - Collations
 - Domains
 - FTS Configurations
 - FTS Dictionaries
 - FTS Parsers
 - FTS Templates
 - Foreign Tables
 - Functions
 - Materialized Views
 - Operators
 - Procedures
 - Sequences
 - Tables (5)
 - airport
 - booking
 - booking_seat
 - flight
 - passenger
 - Trigger Functions
 - Types
 - Views
 - Subscriptions
 - javaslive6

airport_db/postgres@PostgreSQL 17*

airport_db/postgres@PostgreSQL 17

Query Query History

```
1 select * from Passenger;
```

Data Output Messages Notifications

Showing rows: 1 to 5 Page No: 1 of 1

	passenger_passport_number [PK] character varying (20)	passenger_full_name character varying (100)
1	P121	Togzhan Berikbolova
2	P122	Aida Serik
3	P123	Nura Kulaimov
4	P124	Aruzhan Aikyn
5	P125	Dana Kim

✓ Successfully run. Total query runtime: 76 msec. 5 rows affected. ✕

Total rows: 5 Query complete 00:00:00.076 LF Ln 3, Col 1

Object Explorer

- PRC8
- TSIS1
- airport_db
 - Casts
 - Catalogs
 - Event Triggers
 - Extensions
 - Foreign Data Wrappers
 - Languages
 - Publications
 - Schemas (1)
 - public
 - Aggregates
 - Collations
 - Domains
 - FTS Configurations
 - FTS Dictionaries
 - FTS Parsers
 - FTS Templates
 - Foreign Tables
 - Functions
 - Materialized Views
 - Operators
 - Procedures
 - Sequences
 - Tables (5)
 - airport
 - booking
 - booking_seat
 - flight
 - passenger
 - Trigger Functions
 - Types
 - Views
 - Subscriptions
 - javaslive6

airport_db/postgres@PostgreSQL 17*

airport_db/postgres@PostgreSQL 17

Query Query History

```
1 select * from Booking_seat;
```

Data Output Messages Notifications

Showing rows: 1 to 5 Page No: 1 of 1

	booking_id integer	seat_number character varying (10)
1	1	12A
2	2	15B
3	3	7A
4	4	10C
5	5	5A

✓ Successfully run. Total query runtime: 72 msec. 5 rows affected. ✕

Total rows: 5 Query complete 00:00:00.072 LF Ln 1, Col 28

pgAdmin 4 Object Tools Edit View Window Help

pgAdmin 4

Query Query History

```
1 select * from Booking;
```

Data Output Messages Notifications

Showing rows: 1 to 5 Page No: 1 of 1

	booking_id [PK] integer	passenger_passport_number character varying (20)	flight_number character varying (20)	ticket_price numeric (10,2)
1	1	P121	F121	20000.00
2	2	P122	F122	22000.00
3	3	P123	F123	18000.00
4	4	P124	F124	19000.00
5	5	P125	F125	21000.00

✓ Successfully run. Total query runtime: 62 msec. 5 rows affected. ✕

Total rows: 5 Query complete 00:00:00.062 LF Ln 1, Col 23

DDL

1. Create Table

Object Explorer

airport_database/postgres@PostgreSQL 17*

Query Query History

```
71 arriving_airport_id int,
72 departing_gate varchar(50),
73 arriving_gate varchar(50),
74 airline_id int,
75 act_departure_time timestamp,
76 act_arrival_time timestamp,
77 created_at timestamp,
78 updated_at timestamp
79 );
80
81 create table Passengers (
82     passenger_id int,
83     first_name varchar(50),
84     last_name varchar(50),
85     date_of_birth date,
86     gender varchar(50),
87     country_of_citizenship varchar(50),
88     country_of_residence varchar(50),
89     passport_number varchar(20),
90     created_at timestamp,
91     updated_at timestamp
92 );
93
94 create table Security_check (
95     security_check_id int,
96     check_result varchar(20),
97     created_at timestamp,
98     updated_at timestamp,
99     passenger_id int
100 );
```

Data Output Messages Notifications

CREATE TABLE

Query returned successfully in 55 msec.

✓ Query returned successfully in 55 msec. ✕

Total rows: Query complete 00:00:00.055 LF Ln 100, Col 3

2. Define Primary Key

The screenshot shows the DBeaver interface with the Object Explorer on the left and the SQL editor on the right. The Object Explorer shows the 'airport_database' selected, with a list of tables including 'Airline_info', 'Airport', 'Baggage_check', 'Baggage', 'Boarding_pass', 'Booking_flight', 'Booking', 'Flights', 'Passengers', 'Security_check', and 'info'. The SQL editor contains the following queries:

```
1 alter table Airline_info
2 add primary key (airline_id);
3
4 alter table Airport
5 add primary key (airport_id);
6
7 alter table Baggage_check
8 add primary key (baggage_check_id);
9
10 alter table Baggage
11 add primary key (baggage_id);
12
13 alter table Boarding_pass
14 add primary key (boarding_pass_id);
15
16 alter table Booking_flight
17 add primary key (booking_flight_id);
18
19 alter table Booking
20 add primary key (booking_id);
21
22 alter table Flights
23 add primary key (flight_id);
24
25 alter table Passengers
26 add primary key (passenger_id);
27
28 alter table Security_check
29 add primary key (security_check_id);
```

The 'Data Output' tab shows the results of the queries, indicating that the queries were executed successfully. A green status bar at the bottom right confirms: 'Query returned successfully in 49 msec.'

3. Define not null attributes

The screenshot shows the DBeaver interface with the Object Explorer on the left and the SQL editor on the right. The Object Explorer shows the 'public' schema selected, with a list of tables including 'Airline_info', 'Airport', 'Baggage', 'Baggage_check', 'Boarding_pass', 'Booking', 'Flights', 'Passengers', 'Security_check', and 'info'. The SQL editor contains the following queries:

```
52
53 alter table Flights
54 alter column sch_departure_time set not null,
55 alter column sch_arrival_time set not null,
56 alter column departing_airport_id set not null,
57 alter column arriving_airport_id set not null,
58 alter column departing_gate set not null,
59 alter column arriving_gate set not null,
60 alter column airline_id set not null,
61 alter column act_departure_time set not null,
62 alter column act_arrival_time set not null,
63 alter column created_at set not null,
64 alter column updated_at set not null;
65
66 alter table Passengers
67 alter column first_name set not null,
68 alter column last_name set not null,
69 alter column date_of_birth set not null,
70 alter column gender set not null,
71 alter column country_of_citizenship set not null,
72 alter column country_of_residence set not null,
73 alter column passport_number set not null,
74 alter column created_at set not null,
75 alter column updated_at set not null;
76
77 alter table Security_check
78 alter column check_result set not null,
79 alter column created_at set not null,
80 alter column updated_at set not null,
81 alter column passenger_id set not null;
```

The 'Data Output' tab shows the results of the queries, indicating that the queries were executed successfully. A green status bar at the bottom right confirms: 'Query returned successfully in 67 msec.'

4. Rename table

Object Explorer

- Publications
- Schemas (1)
 - public
 - Aggregates
 - Collations
 - Domains
 - FTS Configurations
 - FTS Dictionaries
 - FTS Parsers
 - FTS Templates
 - Foreign Tables
 - Functions
 - Materialized Views
 - Operators
 - Procedures
 - Sequences
 - Tables (10)
 - airline_info
 - Columns (7)
 - airline_id integer
 - airline_code character varying(30)
 - airline_name character varying(50)
 - airline_country character varying(50)
 - created_at timestamp without time zone
 - updated_at timestamp without time zone
 - info character varying(50)
 - Constraints
 - Indexes
 - RLS Policies
 - Rules
 - Triggers
 - airport
 - baggage
 - baggage_check
 - boarding_pass
 - booking

Query

```
1 alter table Airline_info
2 rename to Airline;
```

Data Output Messages Notifications

ALTER TABLE

Query returned successfully in 41 msec.

✓ Query returned successfully in 41 msec. X

Total rows: Query complete 00:00:00.041 LF Ln 2, Col 19

5. Rename column name

Object Explorer

- Publications
- Schemas (1)
 - public
 - Aggregates
 - Collations
 - Domains
 - FTS Configurations
 - FTS Dictionaries
 - FTS Parsers
 - FTS Templates
 - Foreign Tables
 - Functions
 - Materialized Views
 - Operators
 - Procedures
 - Sequences
 - Tables (10)
 - airline_info
 - Columns (7)
 - airline_id integer
 - airline_code character varying(30)
 - airline_name character varying(50)
 - airline_country character varying(50)
 - created_at timestamp without time zone
 - updated_at timestamp without time zone
 - info character varying(50)
 - Constraints
 - Indexes
 - RLS Policies
 - Rules
 - Triggers
 - airport
 - baggage
 - baggage_check
 - boarding_pass
 - booking

Query

```
1 alter table Booking
2 rename column price to ticket_price;
```

Data Output Messages Notifications

ALTER TABLE

Query returned successfully in 47 msec.

✓ Query returned successfully in 47 msec. X

Total rows: Query complete 00:00:00.047 LF Ln 2, Col 37

6. Change data type

The screenshot shows the SQL Server Enterprise Manager interface. On the left, the 'Object Explorer' pane displays the database structure, with the 'public' schema expanded and the 'Tables (10)' folder selected. The 'Flights' table is highlighted. On the right, the 'Query' pane shows the following SQL script:

```
1 alter table Flights
2 alter column departing_gate type text;
```

The 'Messages' pane at the bottom right displays the execution results:

```
ALTER TABLE
Query returned successfully in 22 msec.
```

The status bar at the bottom indicates 'Total rows: Query complete 00:00:00.022' and 'LF Ln 2, Col 20'.

7. Drop the column

The screenshot shows the SQL Server Enterprise Manager interface. On the left, the 'Object Explorer' pane displays the database structure, with the 'public' schema expanded and the 'Tables (10)' folder selected. The 'Airline' table is highlighted. On the right, the 'Query' pane shows the following SQL script:

```
1 alter table Airline
2 drop column info;
```

The 'Messages' pane at the bottom right displays the execution results:

```
ALTER TABLE
Query returned successfully in 57 msec.
```

The status bar at the bottom indicates 'Total rows: Query complete 00:00:00.057' and 'LF Ln 2, Col 18'. A green notification box at the bottom right states: '✓ Query returned successfully in 57 msec. ✕'.

8. Relationship between table

Object Explorer

Publications

Schemas (1)

public

Aggregates

Collations

Domains

FTS Configurations

FTS Dictionaries

FTS Parsers

FTS Templates

Foreign Tables

Functions

Materialized Views

Operators

Procedures

Sequences

Tables (10)

airline_info

airport

baggage

baggage_check

boarding_pass

booking

booking_flight

flights

passengers

security_check

Trigger Functions

Types

Views

Subscriptions

airport_db

javaslive6

postgres

Login/Group Roles

Tablespaces

airport_database/postgres@PostgreSQL 17*

airport_database/postgres@PostgreSQL 17

Query

Query History

14

add foreign key (booking_id)

15

references Booking(booking_id);

16

17

alter table Baggage

18

add foreign key (booking_id)

19

references Booking(booking_id);

20

21

alter table Boarding_pass

22

add foreign key (booking_id)

23

references Booking(booking_id);

24

25

alter table Booking_flight

26

add foreign key (booking_id)

27

references Booking(booking_id);

28

29

alter table Booking_flight

30

add foreign key (flight_id)

31

references Flights(flight_id);

32

33

alter table Flights

34

add foreign key (departing_airport_id)

35

references Airport(airport_id);

36

37

alter table Flights

38

add foreign key (arriving_airport_id)

39

references Airport(airport_id);

40

41

alter table Flights

42

add foreign key (airline_id)

43

references Airline(airline_id);

Data Output

Messages

Notifications

ALTER TABLE

Query returned successfully in 79 msec.

✓ Query returned successfully in 79 msec. ✕

Total rows: Query complete 00:00:00.079 LF Ln 1, Col 1